The **tri-color mark-and-sweep algorithm** is a classic **garbage collection (GC)** technique used in many modern runtimes (like JVM, Go, V8). It improves upon basic mark-and-sweep by enabling **incremental and concurrent garbage collection**.

🧠 Core Idea

Instead of just marking objects as *reachable* or *unreachable*, the algorithm classifies objects into **three colors**:

🎨 The Three Colors

1. **White** ⚪
   → Objects **not yet visited** (potential garbage)
2. **Gray** ⚙️
   → Objects **visited but not fully explored**
   (their references/children are not scanned yet)
3. **Black** ⚫
   → Objects **fully processed**
   (visited + all their references scanned)

🔄 Algorithm Flow

1. Initialization

- All objects start as **white**
- Root objects (stack, globals, registers) are marked **gray**

---

2. Mark Phase (Tri-color traversal)

We repeatedly process gray objects:

Step-by-step:

1. Pick a **gray object**
2. Scan all objects it references
3. For each referenced object:
    - If it's **white → turn it gray**
4. After scanning, mark the current object **black**

🧭 Visualization

```
Initially:
[Root] → Gray
All others → White

Process:
Gray → scan → children become Gray → current becomes Black
```

Key Invariant (VERY IMPORTANT)

> ❗ **No black object should point to a white object**

Why?

- If that happens → we might **miss live objects** → memory corruption

---

3. Sweep Phase

- After marking completes:
    - **White objects = unreachable → garbage**
    - These are reclaimed (freed memory)

⚡ Why Tri-color?

Basic mark-and-sweep pauses the whole program (**stop-the-world**).
Tri-color enables:

✅ Incremental GC

- Run GC in small chunks

- Interleave with application execution

✅ Concurrent GC

- GC runs alongside the program

⚠️ The Problem: Mutations During GC

While GC is running:

- The application might modify object references

Example:

```
Black object → points to White object (new reference)
```

This breaks the invariant!

🛠️ Solution: Write Barriers

To maintain correctness, we use **write barriers**.

Two common strategies:

1. Dijkstra's Write Barrier

- If a **black object points to a white object**
  → make the white object **gray**

2. Steele's Write Barrier

- When modifying references:
  → mark the source object **gray again**

🧩 Putting It All Together

High-level flow:

```
1. All objects → White
2. Roots → Gray
3. While Gray exists:
      - Process → turn Black
      - Children → Gray
4. Sweep White → free memory
```

🚀 Real-world Usage

Used in:

- **JVM (G1, CMS, ZGC, Shenandoah)**
- **Go runtime GC**
- **JavaScript engines (V8, SpiderMonkey)**

🧠 Intuition

Think of it like **graph traversal**:

- White = unvisited nodes
- Gray = nodes in queue
- Black = fully processed nodes

👉 It's essentially **BFS/DFS with safety guarantees for concurrent mutation**

🔥 Interview Insight

If you're asked *why tri-color matters*, say:

- Enables **low pause times**
- Supports **concurrent GC**
- Maintains correctness via **invariant + write barriers**

If you want, I can go deeper into:

- How JVM G1 uses tri-color
- SATB vs incremental update barriers
- Or a step-by-step memory diagram simulation (very useful for interviews)